I. Problem Description and Application

The objective of this project is to develop two machine learning models: Gaussian Naive Bayes and Logistic Regression with gradient-based optimization for predicting the presence of heart disease in patients based on their clinical attributes. Accurate prediction models are crucial for early diagnosis, enabling preventive care and timely medical intervention. Successfully predicting heart disease can significantly improve patient outcomes and reduce healthcare costs associated with cardiovascular diseases.

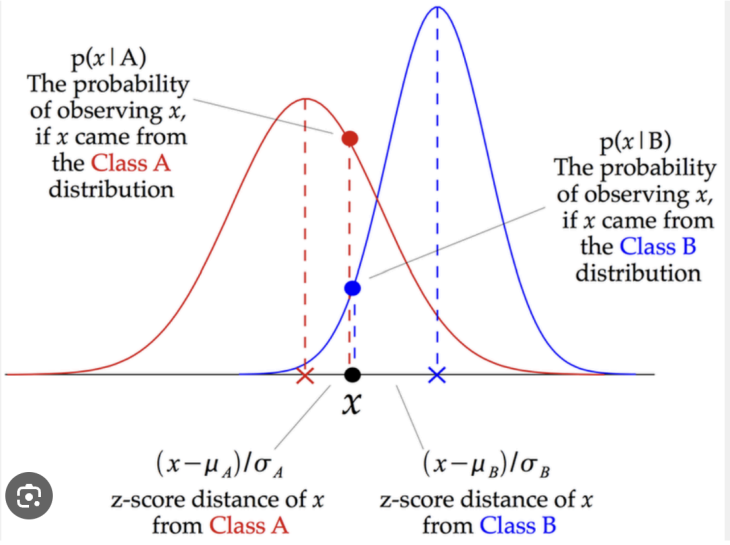
We are using specific metrics to evaluate the accuracy of our methods: Accuracy and True Positive Rate (TPR). Accuracy provides a broad measure of how often the model's predictions are correct across all classes. This is useful for getting an overall sense of the model's performance. On the other hand, TPR, or sensitivity, measures the proportion of actual positives (patients with heart disease) correctly identified by the model. This is crucial in medical diagnosis because the primary goal is to detect as many true cases of heart disease as possible.

II. Related Work and Novelty

Recent studies in heart disease prediction have leveraged diverse machine learning techniques to enhance diagnostic precision. One study employs algorithms such as Random Forest and Logistic Regression, optimized via GridSearchCV, achieving accuracies up to 93.44% [1]. Another regional study in Bangladesh emphasizes the superiority of Random Forest, achieving the highest precision and AUC values, demonstrating its efficacy in clinical settings [2]. Additionally, a novel approach using a concatenated hybrid ensemble classifier demonstrates significant improvements with an accuracy of 86.89% on the UCI dataset, showcasing the benefits of combining multiple models [3]. Furthermore, integrating Decision Trees and SVM offers a tailored solution for complex disease prediction, employing data mining to refine classification accuracy [4]. Our project introduces a novel aspect by integrating probabilistic models like Gaussian Naive Bayes and Logistic Regression, focusing on quantifying uncertainty to provide deeper insights into diagnosis, thereby enhancing both predictive accuracy and interpretability for clinical decision-making. The detailed references include studies on ensemble methods and optimization techniques for improving model performance.

III. Statistical Models and Mathematical Description

1. Gaussian Naive Bayes



Bayes' theorem provides a framework to calculate the probability of an event (class) occurring given some evidence (features): $P(Y|X) = \frac{P(X|Y) \cdot P(Y)}{p(x)}$. Gaussian Naive Bayes assumes features are conditionally independent given the class label. This means the features influence the class decision only through their dependence on the class, and not through any relationships between the features themselves. Mathematically: $P(X|Y) = P(x_1|Y) \cdot P(x_2|Y) \cdot \ldots \cdot P(x_n|Y)$

Gaussian Naive Bayes assumes each feature within a class follows a normal distribution with its mean (μ) and standard deviation (σ). Therefore, the class conditional probability can be expressed as

$$P(x_i|Y) = N(x|\mu_i(Y), \sigma_i^2(Y)) = \frac{1}{\sqrt{2\pi\sigma_i^2(Y)}} \cdot e^{-\frac{(x_i-\mu_i(Y))^2}{2\sigma_i^2(Y)}}$$

The posterior loss, L(Y|X), represents the cost associated with predicting class Y for a data point with features X, when the true class is something else. In the specific case of the 0-1 loss function, minimizing the expected loss over all data points is equivalent to maximizing the probability of correctly classifying each data point.

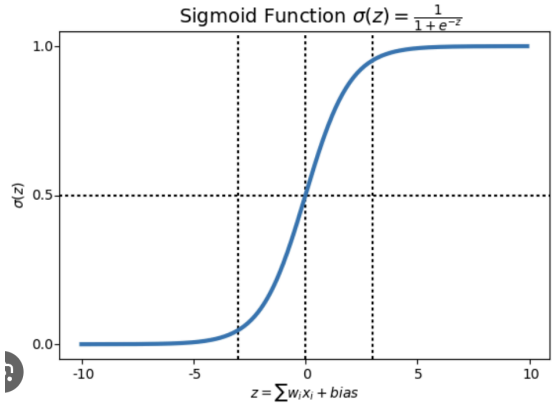
Minimizes the posterior expected loss: $y(x) = argmin_y E[L(t,y)|X = x] = argmin_y \sum_{t=0}^{1} L(t,y)p(t|x)$

Optimal to decide $t = 1$ iff: $\lambda_{01}\, p(t = 0|x) \leq \lambda_{10}\, p(t = 1|x)$ => $\frac{\lambda_{01}\, p(x|t=0)}{p(x)} \leq \frac{\lambda_{10}\,(1-q)p(x|t=1)}{p(x)} \Rightarrow \frac{p(x|t=1)}{p(x|t=0)} \geq \frac{q}{1-q}\frac{\lambda_{01}}{\lambda_{10}}$

(Sudderth, 2024, slide 1)

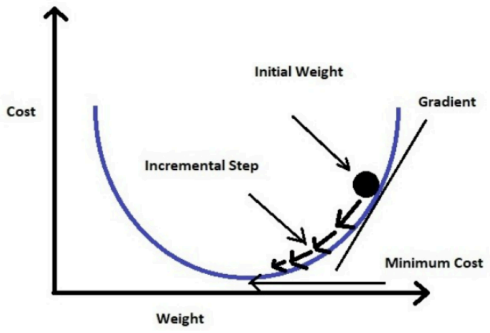
2. Logistic Regression

The core of logistic regression is a linear model that relates the features (X) of a data point to the log-odds of belonging to the positive class (y = 1). This is expressed as: $z = w^T X + b$



The linear model (z) predicts a continuous value. However, in classification, we want a probability between 0 and 1. The sigmoid function, also known as the logistic function, transforms the linear predictor (z) into a probability: $h(z) = \frac{1}{1+e^{-z}}$. This function outputs a value between 0 (when z is negative infinity) and 1 (when z is positive infinity). As z increases, the probability h(z) approaches 1, indicating a higher chance of belonging to the positive class.

The cost function, also known as the loss function, measures how well the model's predictions (probabilities) align with the true labels (0 or 1). A commonly used cost function for logistic regression is the binary cross-entropy: $f(w) = -\log p(t|x,w) = -\sum_{n=1}^{N} t_n log(\mu_n) + (1 - t_n)log(1 - \mu_n)$



Gradient descent is an optimization algorithm used to find the weight vector (w) and bias term (b) that minimize the cost function (L). It iteratively updates the weights and bias in the direction opposite the gradient of the cost function. The gradient tells us how much and in which direction the cost function changes for small changes in the weights and bias.

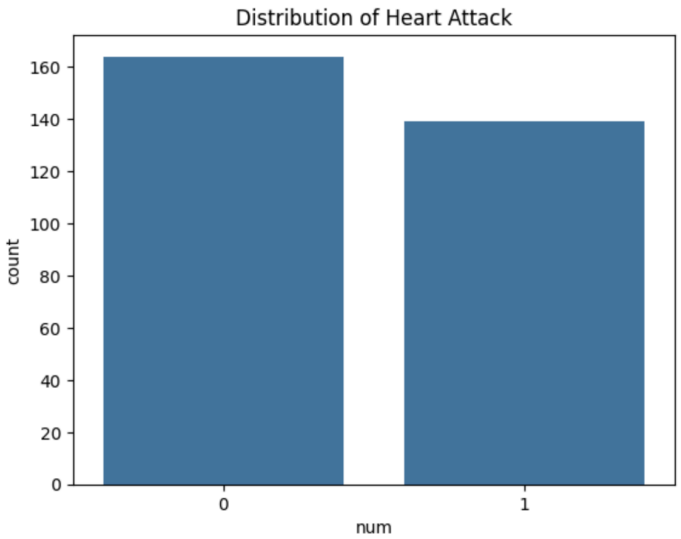
Gradient: $\nabla f(w) = \sum_{n=1}^{N} (\mu_n - t_n)\phi(x_n) = \mathbf{\Phi}^T(\mu - t)$

Optimum: $\frac{1}{N}\sum_{n=1}^{N} t_n\phi(x_n) = \frac{1}{N}\sum_{n=1}^{N} \mu_n\phi(x_n) = \frac{1}{N}\sum_{n=1}^{N} \sigma(w^T\phi(x_n))\phi(x_n)$

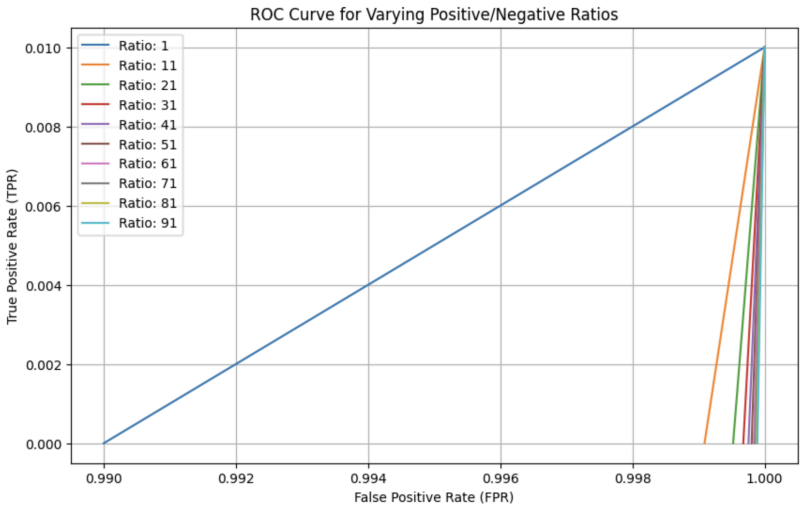
(Sudderth, 2024, slide 4)

IV. Implementation Details

1. Programming Languages and Tools: Our implementation is entirely in Python, utilizing its comprehensive libraries for numerical and statistical analysis. We use **NumPy** for numerical operations and **Pandas** for effective dataset management. **Matplotlib** and **Seaborn** facilitate clear and concise data visualization. **SciPy** helps in optimizing and refining our models, while **Scikit-learn** is crucial for preprocessing data and evaluating model performance. Additionally, **ucimlrepo** allows us to conveniently fetch datasets directly from the UCI Machine Learning Repository, simplifying access to standardized data for analysis.
2. Data Preprocessing: We begin by fetching the Heart Disease dataset from the UCI Machine Learning Repository, ensuring a solid basis for our analysis. After cleaning the dataset by removing rows with missing values, we standardize the features using Scikit-learn's "StandardScaler" to ensure uniform input scales for logistic regression. We also binarize the target variable using a predefined threshold. Finally, we split the dataset into training and testing sets, reserving 20% for testing to validate the model's accuracy.
3. Distribution of Heart Disease Outcomes: The dataset has 5 outcomes, categorial 0 indicates don't have heart disease and categorical 1-4 indicates different kinds of heart disease, so we combine categorical 1-4 to 1. The bar chart illustrates a relatively balanced distribution of heart disease outcomes in the dataset, essential for unbiased model training.



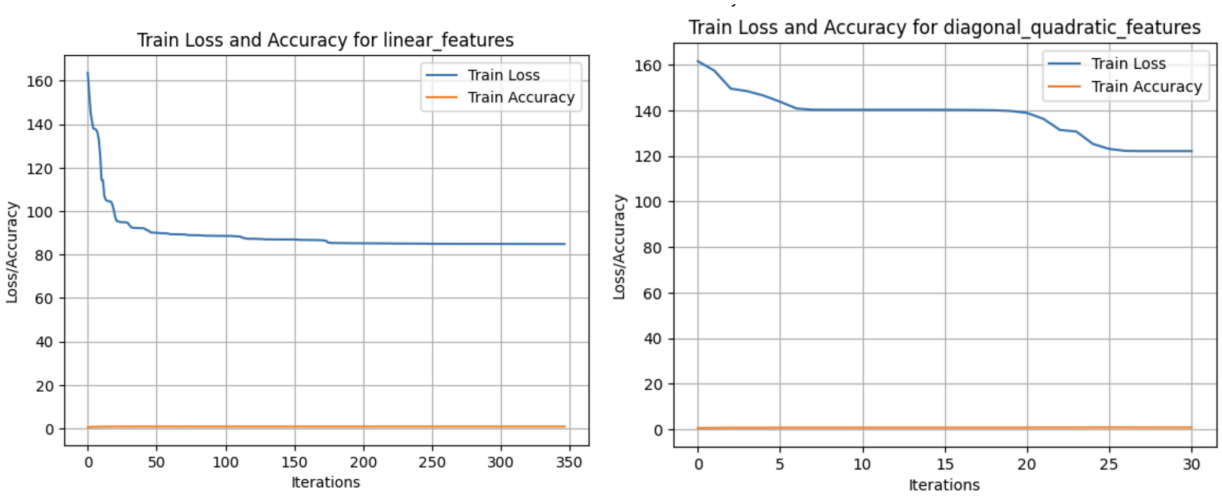
4. Details For Gaussian Naive Bayes: Our implementation focused on calculating the necessary parameters for the Gaussian distributions of each class, and using these parameters to classify data based on the calculated probabilities. Core functionalities implemented include:
    a. Parameter estimation: computes the class priors, means, and variances for each feature within each class. Results: piror0: 0.5232, piror1: 0.4768
    b. Probability computation: applies the Gaussian probability density function to compute the likelihoods of new instances given the class-specific parameters.
    c. Classification: uses these probabilities to classify new instances by calculating log-likelihood ratios and comparing them to a decision threshold.
    d. Performance evaluation: calculates performance metrics such as accuracy, and true positive rate. Result: when all errors are equally costly, accuracy = 0.9167, True Positive Rate (TPR) = 0.8333
    e. Cost modification: Since we want our model to have a higher true positive rate, we modify the class prior ratio, and plot the graph of the ROC curve change:



We find that when the ratio increases, the true positive rate becomes higher but the accuracy(area under the ROC curve) becomes smaller. For example, when ratio = 50, accuracy = 0.7833, True Positive Rate(TPR) = 0.9583. We need to find a balance between accuracy and True Positive Rate.

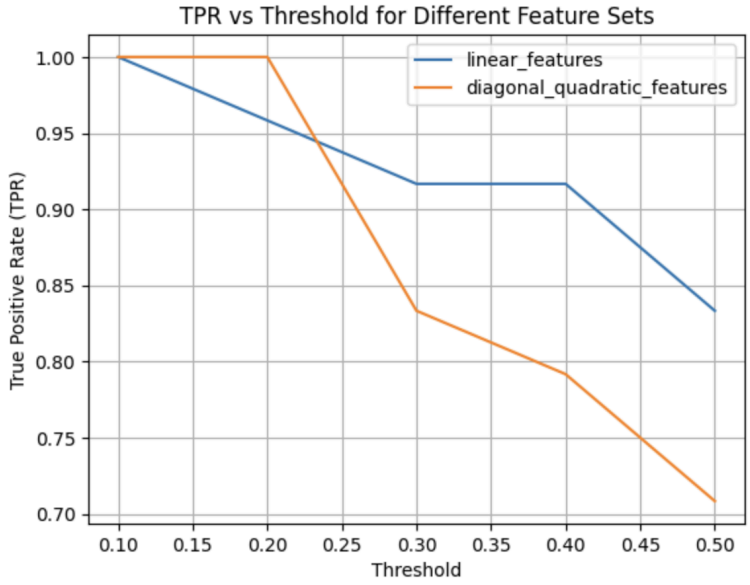
5. Details for Logistic Regression: Our implementation focuses on optimizing model parameters using different feature sets, like linear and diagonal quadratic transformations, to enhance prediction accuracy. We utilize the L-BFGS-B algorithm for efficient optimization, monitoring the process by tracking loss and accuracy. This approach ensures effective model tuning and provides insights into the training dynamics.

a. Feature engineering: prepares data with different feature transformations to explore how each impacts model performance, which includes basic linear features and more complex diagonal quadratic features to potentially capture non-linear relationships.
b. Objective and gradient functions: the objective function (negative log-likelihood with L2 regularization) and its gradient for logistic regression with a given feature function.
c. Optimization with convergence tracking: the model parameters are optimized using the L-BFGS-B algorithm, we track and visualize the loss and accuracy to monitor how the model improves over iterations and ensure it converges effectively throughout the optimization process.
d. Performance evaluation: evaluates the effectiveness using key performance metrics, including calculating accuracy and true positive rate to access the model's predictive capabilities.
Here is the train loss and accuracy change along iterations:



- Linear feature (Left Figure): Train Loss: 84.9, Train Accuracy: 0.835, Test Loss: 19.1, Test Accuracy: 0.867, Test TPR: 0.833
- Diagonal quadratic feature (Right Figure): Train Loss: 122, Train Accuracy: 0.743, Test Loss: 25.2, Test Accuracy: 0.767, Test TPR: 0.708

e. Threshold modification: Since we want our model to have a higher true positive rate, we modify the threshold, and plot the TPR change:



We find that when the threshold decreases, the true positive rate becomes higher but the accuracy(area under the ROC curve) becomes smaller.

For example, when threshold = 0.35:

	Train Loss	Train Accuracy	Test Loss	Test Accuracy	Test TPR
Linear	84.9	0.835	19.1	0.867	0.917
Diagonal Quadratic	122	0.743	25.2	0.767	0.833

We need to find a balance between accuracy and True Positive Rate. In the end, we find out that when the threshold <= 20, the diagonal quadratic feature works better, and when the threshold > 20, the linear feature works better.

V. Contributions of Team Members

Wenqian was mainly responsible for data preprocessing and analysis, managing the retrieval and cleaning of the Heart Disease dataset from the UCI Machine Learning Repository, normalizing feature scales, and splitting the dataset for training and testing. Yufei led model development and optimization, setting up and refining the Gaussian Naive Bayes and Logistic Regression models, with a special emphasis on feature engineering and coding optimization algorithms to improve prediction accuracy. Yu-Han focused on evaluating the models' performance, documenting the accuracy and True Positive Rate, and devising strategies to optimize the trade-off between accuracy and TPR, thus enhancing model sensitivity for potential clinical applications. We compiled and wrote the final project report together, ensuring that all findings, methodologies, and statistical analyses were accurately documented and clearly communicated.

Reference

1. Nadikatla, C., Peddakrishna, S. “Enhancing Heart Disease Prediction Accuracy through Machine Learning Techniques and Optimization.” *Processes*, vol. 11, no. 4, 2023, p. 1210. Available online: https://www.mdpi.com/2227-9717/11/4/1210
2. Hossain, S., Hasan, M. K., Faruk, M. O., Aktar, N., Hossain, R., & Hossain, K. “Machine Learning Approach for Prediction Cardiovascular Disease in Bangladesh.” *BMC Cardiovascular Disorders*, 2024. Available online: https://bmccardiovascdisord.biomedcentral.com/articles/10.1186/s12872-024-03883-2#citeas
3. Majumder, A. B., Gupta, S., Singh, D., Acharya, B., Gerogiannis, V. C., Kanavos, A., & Pintelas, P. "Heart Disease Prediction Using Concatenated Hybrid Ensemble Classifiers." *Algorithms,* vol. 16, no. 12, 2023. Available online: https://www.mdpi.com/1999-4893/16/12/538
4. Saraswathi, R. V., Gajavelly, K., Nikath, A. K., Vasavi, R., & Anumasula, R. R. "Heart Disease Prediction Using Decision Tree and SVM." *Proceedings of Second International Conference on Advances in Computer Engineering and Communication Systems,* 2022. Available online: https://www.researchgate.net/publication/358771007_Heart_Disease_Prediction_Using_Decision_Tree_and_SVM
5. Sudderth, E. (2024). *CS275P: Graphical Models & Statistical Learning Lecture Notes* (slides 1, 4). UC Irvine Department of Computer Science & Statistics.
6. Sudderth, E. (2024). CS275P: Graphical Models & Statistical Learning Homework Codes (homework 1, 3). UC Irvine Department of Computer Science & Statistics.